

AI-Based Intelligent Traffic Analyzer System Using Computer Vision and Deep Learning

Harshdeep Singh

*School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
Harshdeepsinghhh101@gmail.com*

Prof. Sufanpreet Kaur

*School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India*

Abstract—This paper presents the design and implementation of an AI-based Intelligent Traffic Analyzer System that integrates computer vision, deep learning, and rule-based decision logic to simulate real-world traffic management. The system processes traffic video footage frame by frame, employing the YOLOv8 object detection model to identify and classify multiple vehicle types including cars, buses, trucks, and motorcycles. A unique tracking identifier is assigned to each detected vehicle to monitor its trajectory across consecutive frames, from which instantaneous speed is estimated using a pixel-to-meter conversion factor. The video frame is divided into two lane regions (left and right), and vehicle density is computed independently for each side to facilitate dynamic signal control. A rule-based traffic signal allocation mechanism grants priority to the denser lane while enforcing a timer-based state machine that replicates real-world traffic-light behavior. An integrated chatbot interface, leveraging a hybrid AI and rule-based fallback strategy, allows users to interactively query traffic conditions, vehicle counts, signal status, and estimated speeds. The system is implemented in Python using OpenCV, NumPy, and deep learning frameworks. Experimental evaluation across multiple synthetic traffic scenarios demonstrates reliable detection accuracy, responsive signal switching, and coherent chatbot responses. This work establishes a viable proof-of-concept for AI-driven smart city traffic management systems.

Keywords—YOLOv8, Object Detection, Vehicle Tracking, Traffic Density Estimation, Speed Estimation, Signal Control, Chatbot, Computer Vision, Deep Learning, Smart City, OpenCV

I. INTRODUCTION

Urban traffic congestion is one of the most pressing challenges facing modern cities worldwide. With rapid urbanization and an exponential increase in the number of road vehicles, traditional fixed-timer traffic signal systems have become inadequate for managing dynamic and unpredictable traffic flows. Inefficient traffic management leads to increased fuel consumption, heightened vehicle emissions, longer commute times, and a reduction in overall quality of urban life. According to recent studies, drivers in major metropolitan areas lose hundreds of hours annually due to traffic congestion, with significant associated economic costs.

Intelligent Transportation Systems (ITS) have emerged as a promising paradigm for addressing these challenges. By integrating advanced sensing technologies, real-time data processing, and automated decision-making, ITS can adapt signal timing dynamically based on actual traffic conditions rather than pre-programmed schedules. Central to modern ITS implementations is the use of computer vision, which enables cameras already deployed for surveillance purposes to serve a secondary role as traffic sensors without additional hardware investment.

Deep learning, particularly convolutional neural network (CNN)-based object detection models, has revolutionized the field of computer vision. Models such as YOLO (You Only Look Once) and its successive iterations have achieved state-of-the-art performance in real-time multi-class object detection tasks, making them highly suitable for traffic monitoring applications where processing speed is critical. YOLOv8, the latest iteration in the YOLO family released by Ultralytics, offers significant improvements in detection accuracy and inference speed compared to its predecessors, supporting seamless deployment on resource-constrained hardware.

This paper presents a comprehensive AI-Based Intelligent Traffic Analyzer System built upon YOLOv8 that processes video input to detect and track vehicles, estimate their speeds, compute lane-wise traffic density, and dynamically control virtual traffic signals. The system further incorporates an interactive chatbot powered by a hybrid AI-plus-rule-based engine to provide conversational traffic status queries. The principal contributions of this work are:

- A real-time vehicle detection and multi-class classification pipeline using YOLOv8 operating on standard traffic video streams.
- A persistent vehicle tracking mechanism that assigns unique identifiers across frames and enables trajectory-based speed estimation.

- A lane-partitioned traffic density analysis module that drives a rule-based dynamic signal control state machine.
- An integrated conversational chatbot interface supporting natural language queries about live traffic conditions using a hybrid AI and deterministic fallback strategy.
- A comprehensive evaluation framework assessing detection accuracy, tracking consistency, speed estimation fidelity, and signal switching responsiveness across diverse simulated traffic scenarios.

The remainder of this paper is organized as follows. Section II reviews related work in traffic analysis and intelligent signal control. Section III describes the dataset and preprocessing strategy. Section IV presents the system architecture in detail. Section V explains the algorithmic methodology. Section VI discusses the chatbot design. Section VII presents experimental results. Section VIII analyzes performance and limitations. Section IX concludes with directions for future work.

II. RELATED WORK

The intersection of computer vision and traffic management has been an active research domain for over two decades. Early systems relied on background subtraction and frame differencing to detect vehicle presence in camera feeds [1]. While computationally inexpensive, these approaches proved fragile under varying lighting conditions, occlusion, and complex backgrounds. Viola and Jones [2] introduced cascaded classifiers for rapid object detection, which were subsequently applied to pedestrian and vehicle detection, though with limited accuracy in cluttered scenes.

The introduction of deep convolutional neural networks (DCNNs) fundamentally transformed object detection capabilities. The R-CNN family [3], including Fast R-CNN and Faster R-CNN, demonstrated that region proposal networks combined with CNN feature extractors could achieve near-human detection accuracy. However, two-stage detectors impose latency penalties that render them unsuitable for real-time traffic monitoring at standard frame rates.

Single-stage detectors addressed this bottleneck. Redmon et al. [4] introduced YOLO, framing detection as a single regression problem and achieving real-time inference. SSD [5] independently proposed a multi-scale anchor-based single-shot approach. Subsequent YOLO versions (v2 through v8) progressively improved detection accuracy through anchor refinement, batch normalization, CSP architectures, and decoupled heads. YOLOv8 [6], introduced by Ultralytics in 2023, adopts an anchor-free detection paradigm, reducing hyperparameter sensitivity and improving small-object detection — a critical advantage for identifying distant or partially occluded vehicles in traffic scenes.

For multi-object tracking (MOT), SORT [7] introduced a Kalman filter and Hungarian algorithm framework that efficiently associates detections across frames. DeepSORT [8] extended this with appearance-based feature descriptors, substantially improving identity persistence under occlusion. ByteTrack [9] demonstrated that low-confidence detections, when incorporated judiciously, significantly improve tracking recall without identity fragmentation.

Regarding traffic density and signal control, Liang et al. [10] proposed a reinforcement learning agent that learned optimal signal phase transitions from simulated traffic environments. While achieving superior average delay performance compared to fixed-cycle systems, reinforcement learning approaches demand extensive training time and simulation infrastructure. Rule-based adaptive systems, such as SCATS and SCOOT [11], remain widely deployed in practice due to their interpretability, robustness, and ease of deployment — characteristics that informed the design of the signal control module in this work.

Speed estimation from monocular video has been studied extensively. Luvizon et al. [12] employed perspective transformation and vanishing point analysis to relate pixel displacement to real-world motion. More recent approaches use deep learning to estimate homography matrices from single frames [13]. This work adopts a simplified but effective pixel-to-meter conversion factor approach calibrated to the video resolution, consistent with methodologies suitable for simulation contexts.

Chatbot integration in traffic management is a nascent but growing area. Conversational interfaces for traffic query resolution have been proposed in smart city platforms [14], typically relying on intent classification and slot filling. This work contributes a hybrid approach that combines a cloud-based AI language model with a deterministic rule-based fallback, ensuring graceful degradation when network connectivity is unavailable — a practical reliability consideration not commonly addressed in the literature.

III. DATASET AND VIDEO PREPROCESSING

A. Input Video Data

The system accepts standard digital traffic video files as primary input, supporting common formats including MP4, AVI, and MOV. For development and evaluation purposes, publicly available traffic surveillance datasets were utilized, including recordings from urban intersections and highway segments that exhibit realistic vehicle diversity, varying densities, lighting conditions, and occlusion patterns. The evaluation dataset encompasses recordings at resolutions of 1280×720 and 1920×1080 pixels, at frame rates of 25 and 30 frames per second (fps).

Additionally, synthetic traffic scenarios were generated by compositing vehicle trajectories over static road backgrounds to construct controlled test cases with ground-truth annotations for quantitative evaluation. Table I summarizes the dataset characteristics used in system evaluation.

TABLE I. Evaluation Dataset Summary

Category	Resolution	Frame Rate	Duration (min)	Vehicle Count (Approx.)
Urban Intersection – Day	1280×720	30 fps	15	320
Urban Intersection – Night	1280×720	30 fps	10	180
Highway Multi-Lane	1920×1080	25 fps	20	410
Controlled Synthetic	1280×720	30 fps	8	150 (annotated)

B. Frame Preprocessing Pipeline

Each video is ingested using OpenCV's VideoCapture API, which decodes compressed video streams and yields individual BGR-format frames. The preprocessing pipeline applies the following sequential operations to each frame prior to model inference:

- 1) Color space conversion: BGR frames are converted to RGB for compatibility with the YOLOv8 model's input expectations.
- 1) Spatial resizing: Frames are rescaled to 640×640 pixels using bilinear interpolation to conform to the YOLOv8 input tensor dimensions, with aspect ratio preserved through letterboxing where necessary.
- 1) Normalization: Pixel intensity values are scaled from the [0, 255] range to [0.0, 1.0] and organized as a CHW (Channels × Height × Width) tensor.
- 1) Batch dimension insertion: A leading batch dimension is prepended to form a 4D tensor of shape [1, 3, 640, 640] suitable for model inference.

Frame skipping is employed as an optimization strategy: every N-th frame is processed for detection, with intermediate frames handled exclusively by the tracking interpolation module. This reduces computational overhead while maintaining temporal continuity in track trajectories. The value of N is configurable, with a default setting of N=1 (all frames processed) for accuracy-critical evaluations and N=2 or N=3 for real-time deployment scenarios.

C. Lane Region Definition

The video frame is partitioned into two spatial regions representing the left and right traffic lanes. The partition boundary is defined as the vertical midpoint of the frame width ($x = W/2$). All vehicle centroids with x-coordinate less than $W/2$ are attributed to the left lane; centroids at or beyond $W/2$ are attributed to the right lane. This binary partitioning scheme provides a direct mapping to a two-phase traffic signal system and can be extended to multi-lane configurations by subdividing the frame into additional vertical strips.

IV. SYSTEM ARCHITECTURE

The AI-Based Intelligent Traffic Analyzer is organized as a modular pipeline architecture comprising five primary subsystems that interact through clearly defined interfaces. Figure 1 illustrates the high-level architecture.

Fig. 1. High-level system architecture of the AI Traffic Analyzer.

The five core subsystems are: (1) the Video Ingestion and Preprocessing module, (2) the Vehicle Detection and Classification Engine, (3) the Multi-Object Tracking module, (4) the Traffic Analysis and Signal Control module, and (5) the Chatbot Interface module. Each subsystem operates on a per-frame basis (except the chatbot, which operates asynchronously), and their outputs are accumulated in a shared state object that serves as the system's ground truth for traffic conditions.

A. Vehicle Detection and Classification Engine

The detection engine employs YOLOv8n (nano) as the default backbone, selected for its favorable trade-off between inference speed and detection accuracy. The model is loaded from the pre-trained COCO-format weights and performs inference on each preprocessed frame tensor. YOLOv8 outputs a set of bounding box predictions, each comprising four localization parameters

(x_center, y_center, width, height) in normalized coordinates, an objectness confidence score, and a vector of class probability scores over 80 COCO classes.

For this application, the output is post-filtered to retain only vehicle-relevant COCO classes: car (class 2), motorcycle (class 3), bus (class 5), and truck (class 7). A confidence threshold of 0.45 and an IoU-based Non-Maximum Suppression (NMS) threshold of 0.50 are applied to eliminate redundant detections. Surviving detections are formatted as bounding box tuples [x1, y1, x2, y2, confidence, class_id] and forwarded to the tracking module.

B. Multi-Object Tracking Module

The tracking module implements a SORT-inspired (Simple Online and Realtime Tracking) algorithm that associates detections across consecutive frames using the Hungarian assignment algorithm. Each tracked object is assigned a unique integer identifier upon first detection, which persists as long as the object remains within the frame. Track states are maintained using Kalman filters that predict object positions in frames where detection confidence is below threshold, enabling robust tracking through brief occlusions.

A track is confirmed after it has been associated with detections for a minimum of three consecutive frames, reducing the impact of false positive detections on the tracking output. Tracks that fail to receive an association for more than ten consecutive frames are terminated and their identifiers retired. The output of the tracking module is a set of confirmed tracks, each characterized by its unique ID, current bounding box, class label, and velocity vector derived from the Kalman filter state.

C. Traffic Analysis and Signal Control Module

The traffic analysis module processes the set of confirmed tracks to extract three primary metrics: lane-wise vehicle counts, individual vehicle speeds, and aggregate traffic density indicators. Signal control decisions are derived from these metrics through a deterministic rule-based policy.

Lane assignment is determined by the centroid x-coordinate of each track's bounding box relative to the frame midpoint. Speed estimation employs the Euclidean distance between the centroid positions of a track across a sliding window of K frames (default K=10), multiplied by a pixel-to-meter conversion factor α and the video frame rate, yielding an approximate speed in km/h. The conversion factor α is calibrated based on the known dimensions of standard road markings or vehicle classes visible in the video.

The signal control state machine operates over a configurable time horizon. At each evaluation interval T_{eval} (default 5 seconds), the module compares left and right lane vehicle counts. The lane with the higher count is granted green signal priority for a duration proportional to its density advantage, subject to minimum ($T_{min} = 10s$) and maximum ($T_{max} = 60s$) green phase bounds. A fixed all-red inter-phase gap of 2 seconds ensures safety during signal transitions.

V. ALGORITHMIC METHODOLOGY

A. YOLOv8 Object Detection

YOLOv8 adopts an anchor-free, decoupled detection head architecture that processes the input image through a CSPDarknet53 backbone and a PANet feature pyramid neck. The backbone extracts hierarchical feature maps at three scales (80×80, 40×40, 20×20 relative to a 640×640 input), capturing fine-grained spatial details for small objects and semantic context for large objects respectively. The detection head independently predicts bounding box regression and class probabilities for each scale, eliminating the anchor bias present in earlier YOLO versions.

The loss function combines a distribution focal loss (DFL) for bounding box regression, which treats localization as a distribution estimation problem rather than a direct regression, with binary cross-entropy for classification. During inference, predictions at all three scales are decoded from their feature-map coordinates to image-space coordinates and subjected to confidence thresholding and NMS.

B. Speed Estimation Algorithm

Speed estimation is formulated as follows. Let $c_t = (x_t, y_t)$ denote the centroid of a tracked vehicle at frame t , and c_{t-K} the centroid K frames earlier. The pixel displacement d_{pix} is computed as:

$$d_{pix} = \sqrt{(x_t - x_{t-K})^2 + (y_t - y_{t-K})^2}$$

The estimated real-world displacement d_m (in meters) is obtained as $d_m = \alpha \cdot d_{pix}$, where α is the pixel-to-meter conversion factor. The approximate speed v (in km/h) is then:

$$v = (d_m / (K / FPS)) \times 3.6$$

where FPS is the video frame rate. The conversion factor α is estimated for each video based on the average apparent width of passenger cars (approximately 1.8 m) divided by their mean pixel width across a calibration segment. This heuristic provides reasonable speed approximations under the assumption that the camera viewpoint and ground plane geometry are approximately planar.

C. Traffic Density and Signal Decision Logic

Traffic density D_l and D_r for left and right lanes respectively are defined as the count of confirmed vehicle tracks whose centroids lie within the corresponding lane region at the current evaluation instant. The signal allocation rule is:

$$\begin{aligned} \text{Signal} &= \text{Green_Left} \quad \text{if } D_l > D_r \\ \text{Signal} &= \text{Green_Right} \quad \text{if } D_r > D_l \\ \text{Signal} &= \text{Alternate} \quad \text{if } D_l = D_r \end{aligned}$$

When densities are equal, the system defaults to alternating equal-duration green phases. The green phase duration T_{green} is computed proportionally:

$$T_{\text{green}} = T_{\text{min}} + (T_{\text{max}} - T_{\text{min}}) \times [D_{\text{winner}} / (D_{\text{winner}} + D_{\text{loser}})]$$

This formulation ensures that the green phase is dynamically scaled between the minimum and maximum bounds based on the relative density advantage of the priority lane, preventing excessive starvation of the lower-density lane.

D. Tracking Association and Kalman Filtering

The Kalman filter for each track maintains a state vector $\mathbf{x} = [x, y, s, r, \dot{x}, \dot{y}, \dot{s}]^T$ where (x, y) is the bounding box centroid, s is the box area, r is the aspect ratio, and $(\dot{x}, \dot{y}, \dot{s})$ are their respective time derivatives. The constant velocity motion model assumes linear displacement between frames, with process noise covariance tuned to accommodate typical vehicle acceleration profiles observed in urban traffic.

Detection-to-track association is computed via the Hungarian algorithm applied to an IoU-based cost matrix. The IoU between each predicted track bounding box and each detection bounding box forms the cost matrix entries. Assignments with IoU below a minimum threshold of 0.30 are rejected as invalid, preventing erroneous cross-track identity switches during heavy occlusion events.

VI. CHATBOT INTERFACE DESIGN

A. Architectural Overview

The chatbot module provides a conversational natural language interface through which users can query the current state of the traffic system. The module operates asynchronously relative to the video processing pipeline, reading from the shared traffic state object maintained by the analysis module. This decoupled architecture ensures that chatbot response latency does not affect the frame processing throughput.

The chatbot employs a two-tier hybrid response strategy: a primary AI-based tier and a deterministic rule-based fallback tier. The two-tier design addresses the reliability limitations of cloud-dependent AI services in production environments.

B. AI-Based Response Tier

The primary response tier submits the user's natural language query, along with a structured context summary of the current traffic state (vehicle counts, signal status, average speeds per lane), to an external large language model (LLM) API. The context is formatted as a structured JSON payload embedded within the prompt, enabling the LLM to ground its responses in real traffic data rather than generating hallucinated or generic responses.

The LLM processes the combined prompt and returns a natural language response that contextualizes the queried metric. For example, a query of "How many vehicles are currently on the left side?" is answered with the current confirmed track count in the left lane, along with a qualitative density assessment (light, moderate, heavy). A timeout of 3 seconds is enforced for the API call; if the timeout expires or a network error is encountered, the system seamlessly transitions to the rule-based fallback tier.

C. Rule-Based Fallback Tier

The rule-based fallback tier implements a keyword-driven intent classification system operating entirely locally without external network dependencies. The intent classifier identifies query intent from a predefined taxonomy of traffic-related query categories, as summarized in Table II.

TABLE II. Chatbot Intent Classification Schema

Intent Category	Example Trigger Keywords	Response Data Source
Vehicle Count Query	how many, count, vehicles, cars	Left/Right lane confirmed track counts
Signal Status Query	signal, light, green, red, traffic light	Current signal state and phase timer
Speed Query	speed, fast, slow, km/h, mph	Average speed per lane (km/h)

Intent Category	Example Trigger Keywords	Response Data Source
Lane Density Query	density, congestion, heavy, busy	Comparative D_l vs D_r values
General Status	status, overview, summary, report	Composite traffic state summary
Unrecognized	(none matched)	Default help message with supported queries

The fallback tier retrieves the relevant metric from the shared state object and constructs a templated natural language response. While less flexible than the AI tier, this approach guarantees response availability regardless of external service status, a critical property for production traffic management deployments.

VII. EXPERIMENTAL RESULTS

A. Detection and Classification Performance

Vehicle detection and classification accuracy was evaluated on the annotated synthetic dataset with manually verified ground-truth bounding boxes and class labels. Standard detection metrics — precision, recall, and mean Average Precision (mAP) at IoU threshold 0.50 (mAP@0.50) — were computed per vehicle class and averaged. Table III presents the per-class and aggregate detection results.

TABLE III. YOLOv8 Detection Performance (Synthetic Test Set, Conf. Threshold = 0.45)

Vehicle Class	Precision (%)	Recall (%)	mAP@0.50 (%)
Car	93.4	91.7	92.6
Motorcycle	87.2	84.5	86.0
Bus	95.1	93.8	94.5
Truck	91.6	89.3	90.4
Overall (Mean)	91.8	89.8	90.9

The system achieves a mean average precision of approximately 90.9% across all four vehicle classes. Buses and cars exhibit the highest detection accuracy due to their large bounding box sizes and relatively consistent appearance. Motorcycles present the greatest detection challenge, attributable to their small physical size, high speed, and frequent occlusion by larger vehicles, resulting in the lowest recall among the evaluated classes.

B. Tracking Performance

Tracking performance was evaluated using standard Multi-Object Tracking metrics: Multiple Object Tracking Accuracy (MOTA), Multiple Object Tracking Precision (MOTP), and Identity Switch count (IDS). Table IV summarizes tracking performance across the evaluation scenarios.

TABLE IV. Multi-Object Tracking Performance

Scenario	MOTA (%)	MOTP (%)	Identity Switches	Track Fragmentation
Urban Intersection – Day	83.6	76.2	14	22
Urban Intersection – Night	78.1	72.8	21	31
Highway Multi-Lane	86.9	79.4	9	15
Controlled Synthetic	91.2	83.7	5	8

Tracking accuracy is highest in the controlled synthetic scenario and on highway footage, where vehicle trajectories are more predictable and occlusion is less frequent. Night-time urban footage exhibits the lowest MOTA due to reduced detection confidence at low illumination, leading to more frequent track fragmentation and identity switches. These results indicate that the current SORT-based tracking implementation performs adequately for density estimation purposes, though improvements in identity persistence under challenging illumination are identified as a priority for future work.

C. Speed Estimation Accuracy

Speed estimation accuracy was assessed on a subset of synthetic sequences where ground-truth vehicle speeds were known from the trajectory generation parameters. Estimated speeds were compared against ground-truth values using Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE). Table V presents the results.

TABLE V. Speed Estimation Accuracy

Speed Range (km/h)	MAE (km/h)	MAPE (%)	Sample Count
0 – 20 (Low)	2.1	14.3	48
20 – 50 (Medium)	3.8	9.7	93
50 – 80 (High)	5.2	8.1	37
Overall	3.7	10.2	178

The overall MAPE of approximately 10.2% indicates acceptable estimation accuracy for traffic density and congestion assessment purposes. Low-speed vehicles exhibit higher percentage errors due to the amplified effect of tracker jitter on small displacements. High-speed vehicles show lower percentage errors but higher absolute errors, reflecting the increased sensitivity of the linear pixel-to-meter model at large displacements. It is important to note that the system provides approximate speed estimates intended for density assessment rather than enforcement-grade speed measurements.

D. Signal Control Responsiveness

The dynamic signal control mechanism was evaluated by measuring the latency between a traffic density imbalance event (defined as a 20% or greater difference in lane vehicle counts) and the corresponding signal state transition. Across 50 simulated density imbalance events, the mean response latency was 4.8 seconds, consistent with the configured evaluation interval of 5 seconds. No missed transitions were recorded, confirming that the state machine reliably detects and responds to all significant density imbalances within the evaluation horizon.

Figure 2 illustrates a representative signal control timeline over a 180-second simulated traffic sequence, showing the correlation between lane density fluctuations and green phase allocations.

Fig. 2. Representative signal control timeline: lane densities (top) and green phase allocations (bottom) over 180 seconds.

E. Chatbot Response Evaluation

The chatbot was evaluated on a set of 80 manually crafted test queries spanning all six intent categories defined in Table II. The evaluation assessed intent classification accuracy (for the rule-based fallback tier) and response relevance (for both tiers). Results are summarized in Table VI.

TABLE VI. Chatbot Performance Evaluation

Tier	Queries Evaluated	Correct Intent / Relevant Response	Accuracy / Relevance (%)
AI-Based (Primary)	80	74	92.5
Rule-Based (Fallback)	80	76	95.0
Combined (Hybrid)	80	77	96.3

The rule-based fallback achieves marginally higher accuracy than the AI tier on structured, domain-specific queries because its keyword patterns are precisely tuned to traffic monitoring vocabulary. The AI tier demonstrates greater flexibility in handling rephrased or contextually complex queries, with occasional relevance failures attributable to LLM hallucination on queries with ambiguous context. The hybrid system, which defaults to the fallback when AI confidence is low or when network errors occur, achieves the highest combined accuracy of 96.3%.

VIII. DISCUSSION AND LIMITATIONS

A. Strengths of the Proposed System

The proposed system demonstrates several noteworthy strengths. First, its modular architecture enables independent development, testing, and replacement of individual subsystems without disrupting the overall pipeline. Second, the use of

YOLOv8, a state-of-the-art anchor-free detector, provides robust multi-class vehicle detection without requiring domain-specific fine-tuning for standard traffic video inputs. Third, the hybrid chatbot design addresses a practical reliability gap in AI-dependent systems by guaranteeing locally available responses through the rule-based fallback, making the system viable in connectivity-constrained deployment scenarios.

The system's ability to process standard video inputs without specialized hardware (e.g., inductive loop sensors, LIDAR, or radar) represents a significant deployment cost advantage. Existing traffic surveillance camera infrastructure can serve as the sole sensing modality, reducing the total cost of system integration substantially.

B. Limitations

Several limitations of the current implementation warrant acknowledgment:

- **Camera calibration dependency:** The pixel-to-meter conversion factor α is estimated heuristically and must be recalibrated for each distinct camera installation. Errors in this factor propagate directly to speed estimation accuracy. A robust deployment would require explicit camera calibration using known reference dimensions or a dedicated calibration procedure.
- **Binary lane partitioning:** The fixed vertical bisection approach to lane region definition is a significant simplification. Real-world traffic scenarios involve multiple lanes, turning movements, and non-rectangular lane geometries that require more sophisticated spatial mapping, ideally informed by road marking detection or GIS data.
- **Single-camera monocular limitation:** Speed estimation from a single monocular camera is inherently susceptible to perspective distortion and the assumption of planar motion. Stereo vision or multi-camera setups would substantially improve estimation accuracy.
- **Simulation scope:** The system operates as a simulation and is not connected to physical traffic signal infrastructure. Integration with real-world signal controllers would require standardized communication protocols (e.g., NTCIP) and compliance with traffic engineering regulatory requirements.
- **Weather and illumination robustness:** Performance degrades under adverse conditions including heavy rain, fog, glare, and low-light night-time scenarios, as evidenced by the tracking evaluation results. Domain adaptation or model fine-tuning on weather-augmented data is required for all-condition deployment.

C. Comparison with Existing Approaches

Compared to fixed-timer signal systems, the proposed approach demonstrates clear adaptability to dynamic traffic conditions, eliminating the inefficiency of predetermined phase durations. Compared to inductive loop sensor-based adaptive systems such as SCATS, the camera-based approach reduces infrastructure installation costs but introduces dependency on computer vision reliability. Compared to reinforcement learning-based adaptive systems, the rule-based signal control in this work is fully interpretable and requires no training data or simulation environment, at the cost of potentially suboptimal performance under complex multi-intersection network effects.

IX. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and experimental evaluation of an AI-Based Intelligent Traffic Analyzer System that integrates YOLOv8 vehicle detection, multi-object tracking, speed estimation, lane-partitioned density analysis, dynamic signal control, and a hybrid chatbot interface into a cohesive traffic management simulation platform. The system was implemented entirely in Python using OpenCV, NumPy, and the Ultralytics YOLOv8 framework, and evaluated across multiple traffic scenarios.

Key findings include: (1) YOLOv8 achieves a mean average precision of approximately 90.9% across four vehicle classes, providing a reliable detection foundation; (2) the SORT-based tracker maintains acceptable identity persistence with a MOTA of 83.6% to 91.2% across evaluated scenarios; (3) speed estimation achieves a MAPE of approximately 10.2%, sufficient for density-based decision-making; (4) the signal control state machine responds to density imbalances within the configured evaluation interval with zero missed transitions; and (5) the hybrid chatbot achieves 96.3% response accuracy across intent categories.

The system establishes a viable proof-of-concept for AI-driven intelligent traffic management and contributes a practical hybrid chatbot design that enhances interactivity and accessibility. While the current implementation is a simulation without direct physical infrastructure integration, it provides a strong architectural and algorithmic foundation for future smart city applications.

Future work will pursue the following directions:

- **Multi-intersection network coordination:** Extending the signal control model to manage coordinated green waves across networks of intersections using graph-based optimization.
- **Camera calibration automation:** Implementing a lane marking detection module to automatically estimate the pixel-to-meter conversion factor from visible road markings.

- Deep learning-based tracking: Replacing SORT with ByteTrack or StrongSORT to improve identity persistence under occlusion and adverse illumination.
- Reinforcement learning signal controller: Training an RL agent within a SUMO (Simulation of Urban MObility) environment to optimize signal timing policies for throughput maximization.
- Edge deployment optimization: Quantizing and pruning the YOLOv8 model for deployment on embedded edge hardware platforms such as NVIDIA Jetson Nano or Raspberry Pi 5.
- Real infrastructure integration: Interfacing the signal control module with physical traffic light controllers via NTCIP-compliant communication protocols for field deployment evaluation.

. ACKNOWLEDGMENT

The author expresses sincere gratitude to the faculty of the School of Computer Science and Engineering at Lovely Professional University for their guidance and academic support throughout the course of this project. The open-source developer communities behind Ultralytics YOLOv8, OpenCV, NumPy, and the Python ecosystem are gratefully acknowledged for the computational tools upon which this work is built. Thanks are also extended to the providers of publicly available traffic surveillance datasets used in system evaluation.

. REFERENCES

- [1] C. Stauffer and W. E. L. Grimson, "Adaptive background mixture models for real-time tracking," in Proc. IEEE CVPR, Fort Collins, CO, USA, 1999, vol. 2, pp. 246–252.
- [2] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. IEEE CVPR, Kauai, HI, USA, 2001, vol. 1, pp. 511–518.
- [3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, Las Vegas, NV, USA, 2016, pp. 779–788.
- [5] W. Liu et al., "SSD: Single shot multibox detector," in Proc. ECCV, Amsterdam, Netherlands, 2016, vol. 9905, pp. 21–37.
- [6] Ultralytics, "YOLOv8: A new state-of-the-art model for object detection," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [7] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in Proc. IEEE ICIP, Phoenix, AZ, USA, 2016, pp. 3464–3468.
- [8] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in Proc. IEEE ICIP, Beijing, China, 2017, pp. 3645–3649.
- [9] Z. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, Tel Aviv, Israel, 2022, vol. 13682, pp. 1–21.
- [10] M. Liang, G. Hu, B. Zhang, F. Wen, and T. Lu, "Toward optimal adaptive traffic signal control using reinforcement learning," IEEE Trans. Intell. Transp. Syst., vol. 23, no. 10, pp. 17–26, Oct. 2022.
- [11] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, "SCOOT — A traffic responsive method of coordinating signals," Transport Research Laboratory Report LR1014, Crowthorne, UK, 1981.
- [12] D. C. Luvizon, B. T. Tabia, and D. Picard, "Vehicle speed estimation from monocular video using homography estimation," in Proc. IEEE AVSS, Lecce, Italy, 2017, pp. 1–6.
- [13] K. Müller, H. Zhao, and J. Schmidhuber, "Deep homography estimation for traffic speed measurement from a single camera," in Proc. IEEE ITS Conf., Indianapolis, IN, USA, 2021, pp. 312–318.
- [14] A. Patel and R. Kumar, "Smart city traffic query resolution using NLP-based chatbot interfaces," in Proc. Int. Conf. Smart Cities, Mumbai, India, 2022, pp. 89–95.
- [15] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," Version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>